CSCI2010U MIDTERM - SIDE A: CODE YOU CAN WRITE, TRACES, RECURSION, SEARCHING

HOW TO ANSWER

Print / return? Make a table: one row per pass or call, one column per variable. Run it line by line.
Fill a blank: copy the matching template; check start, stop test, step, names.
Write a method: 1) what to return, 2) variable (count / sum / min), 3) loop or recursion, 4) return AFTER the loop. Test it on the example.

JAVA ESSENTIALS

Array: index from **0**; `a.length` (no brackets); last = `a[a.length-1]`; `a[a.length]` crashes. `int[] a = new int[5];` (all 0) or `{5,2,9}`. Size is fixed.
String: `s.length()`, `s.charAt(i)`, `s.substring(1)` ("hello" gives "ello"), `s.substring(1,3)` ("el", end not included), `s1.compareTo(s2)` (negative = s1 first). Case matters.
`n%10` = last digit, `n/10` drops it (5072: 2 and 507). `==` compares, `=` assigns. Defaults: int 0, boolean false, object null.
Primitives are passed as a COPY. Arrays and objects pass a reference copy, so a method changes the SAME array (no return needed).
Compile error (javac refuses): missing ;, wrong type, private field used outside its class, `new` on an abstract class, parent object put in a child variable, abstract method not written, `return null` in an int method.
Runtime error (crashes while running):
Array IndexOutOfBounds, StackOverflowError.
`i % k == 0` means divisible by k. Integer division truncates: 7 / 2 = 3, 7 % 2 = 1.
A `static` method cannot use instance fields or `this`; an object needs `new` before you use it.
Strings are not arrays: `s[i]` does not compile, use `s.charAt(i)`. `+=` works on Strings, not arrays. An int method returns a number (or throws), never null. Out of range: arrays throw ArrayIndexOutOfBounds, the L11 list throws IndexOutOfBounds.
`javac F.java` makes F.class; `java F` runs `public static void main(String[] args)`.

COUNT, MIN, SUM, MULTIPLES (Lab 1)

```
int count = 0;                  // before the loop
for (int i = 0; i < a.length; i++)
  if (a[i] >= target) count++;  // >= : equal counts
return count;                   // after the loop
countAtLeast({25,10,40,25}, 25):
 i=0 25>=25 yes c=1 | i=1 10>=25 no c=1
 i=2 40>=25 yes c=2 | i=3 25>=25 yes c=3 -> 3
count > 4 in {5,2,9,2}: yes,no,yes,no -> 2
int min = a[0];                 // not 0
for (int i = 1; i < a.length; i++)
  if (a[i] < min) min = a[i];
int sum = 0;
for (int i = 0; i < a.length; i++) sum += a[i];
for (int i=start; i<=end; i++)    // multiples
  if (i % mult == 0) System.out.println(i);
```

REVERSE

```
void reverseArray(int[] a){       // in place
  int l = 0, r = a.length-1;
  while (l < r){ int t=a[l]; a[l]=a[r]; a[r]=t;
    l++; r--; } }        // {1,2,3,4} -> 4 3 2 1
String reverse(String s){ String out = "";
  for (int i=s.length()-1; i>=0; i--)
    out += s.charAt(i);
  return out; }          // "abc" -> "cba"
```

ORDER THE STEPS

```
insert val at idx: check range (return)
  -> i = a.length-1 -> while i>idx: a[i]=a[i-1], i--
  -> a[idx] = val
binary search: low>high return NOT_FOUND -> mid
  -> equal? return mid -> else recurse on a half
TOH: if n==0 return -> TOH(n-1,start,temp,goal)
  -> print start --> goal -> TOH(n-1,temp,goal,start)
bubble: i from last down to 1 -> j from 0 to i-1
  -> if A[j+1] < A[j] swap
delete: check range -> i = idx+1
  -> while i<length: a[i-1]=a[i] -> last = GAP
```

SLIPS TO AVOID (caught in practice)

`i <= a.length` crashes; use `i < a.length`.
Counter starts BEFORE the loop; `count++` goes INSIDE the `if`; `return` goes AFTER the loop.
Bubble sort: `i` is fixed during a pass, `j` moves. n elements: i starts at n-1, n-1 passes.
A recursive call returns a VALUE: `sumTo(2)` is 3, not 2. Each call adds only its own piece (1 per digit, not n).
Text needs quotes and exact case: `"Meow"`. Strings use `compareTo`; numbers use `Integer.compare`.

BUBBLE SORT (L1, L2)

```
for (int i = A.length-1; i > 0; i--)  // i fixed
  for (int j = 0; j < i; j++)         // j moves
    if (A[j+1] < A[j]) swap(A, j, j+1);
swap: int t=A[j]; A[j]=A[k]; A[k]=t;  // needs temp
descending: use A[j+1] > A[j]
{4,2,3}: i=2: j=0 2<4 swap 2 4 3
              j=1 3<4 swap 2 3 4
         i=1: j=0 3<2? no  2 3 4   (2 passes)
{3,1,2}: i=2: 1 3 2 | 1 2 3 ; i=1: 2<1? no
{5,2,4,1}: 2541 | 2451 | 2415 (5 locked)
  pass2: 2415 | 2145 | pass3: 1245
{8,1,6,4,9}: i=4: j0 swap 1 8 6 4 9, j1 swap
  1 6 8 4 9, j2 swap 1 6 4 8 9, j3 9<8? no
  i=3: j0 no, j1 swap 1 4 6 8 9, j2 no
  i=2: no, no | i=1: no  -> 1 4 6 8 9
  (4+3+2+1 = 10 compares)
{1,2,3} already sorted: 3 compares, 0 swaps
{3,2,1} reversed: 3 compares, 3 swaps -> 1 2 3
```

n	i starts	passes	pass1 compares	j max
4	3	3	3	2
5	4	4	4	3
6	5	5	5	4

Total compares = n(n-1)/2. Worst case O(n^2).

SEARCHING (L6, L10)

```
linear: for (int i=0; i<A.length; i++)
          if (A[i]==val) return i;
        return -1;      // best O(1), worst O(n)
binary (array SORTED): bs(a,val,low,high)
  if (low > high) return NOT_FOUND;
  int mid = (low+high)/2;
  if (val == a[mid]) return mid;
  else if (val < a[mid])
    return bs(a,val,low,mid-1);
  else return bs(a,val,mid+1,high);
start: bs(a,val,0,a.length-1)    O(log n)
a={1,3,5,7,9,11,13}
find 11: l0 h6 m3=7, go right: l4
         l4 h6 m5=11 found, return 5
find 3 : l0 h6 m3=7, go left: h2
         l0 h2 m1=3 found, return 1
find 5 : m3=7 h2 | m1=3 l2 | l2 h2 m2=5 -> 2
find 4 : m3=7 h2 | m1=3 l2 | m2=5 h1 | low>high
         NOT_FOUND
```

ARRAY INSERT / DELETE (L10)

```
insert(a,idx,val):
  for (int i=a.length-1; i>idx; i--)
    a[i] = a[i-1];     // shift RIGHT from the end
  a[idx] = val;        // last element falls off
{4,6,8,10} idx=1 val=50:
 4 6 8 8 | 4 6 6 8 | 4 50 6 8   (2 shifts)
delete(a,idx):
  for (int i=idx+1; i<a.length; i++)
    a[i-1] = a[i];     // shift LEFT
  a[a.length-1] = GAP;
{5,6,7,8} idx=1:
 5 7 7 8 | 5 7 8 8 | 5 7 8 GAP  (2 shifts)
worst idx=0 O(n); best idx=last O(1); a[i] O(1)
```

PERFECT SQUARE (L6, L9)

```
v0: while (i <= n) { if (i*i==n) Yes; i++ }  O(n)
v1: while (i*i <= n)                         O(sqrt n)
v2: double i till i*i>n, then binary search O(log n)
n=10: v0 10 loops No | v1 i=1,2,3 (1,4,9), 16>10
      stop = 3 loops, No
n=20: v0 20 loops    | v1 i=1..4 (16), 25>20
      stop = 4 loops, No
n=16: v1 i=1..4, 16==16 Yes (4 loops)
```

MORE WHOLE METHODS (Canvas style)

```
boolean isSorted(int[] a){
  for (int i = 1; i < a.length; i++)
    if (a[i-1] > a[i]) return false;   // not >=
  return true; }     // empty / 1 element = true
int countDivisible(int lo, int hi, int k){
  int count = 0;
  for (int i = lo; i <= hi; i++)
    if (i % k == 0) count++;
  return count; }    // (1, 11, 3) = 3
class Roster { private String[] names; private int size;
  public Roster(int capacity){
    names = new String[capacity]; size = 0; }
  public void add(String name){
    if (size == names.length)
      throw new IndexOutOfBoundsException("full");
    names[size] = name; size++; }
  public String get(int index){
    if (index < 0 || index >= size)
      throw new IndexOutOfBoundsException("bad");
    return names[index]; }
  public int size(){ return size; } }
```

RECURSION (L7, Lab 4)

Needs a **base case** (smallest problem, returns) and a **recursive case** (calls itself on a SMALLER problem). No base case = StackOverflowError.
Trace: go DOWN (calls wait), then back UP (each returns using the value from below). `sumTo(2)` means the SUM 1+2 = 3, not the number 2. Countdown n..0 = n+1 calls.
Print BEFORE the call = going down (3,2,1). Print AFTER = coming up (1,2,3).

```
sumTo(n): if (n==0) return 0;
          return n + sumTo(n-1);
 (0)=0 (1)=1+0=1 (2)=2+1=3 (3)=3+3=6  4 calls
twice(n): if (n==0) return 0;
          return 2 + twice(n-1);
 (1)=2 (2)=4 (3)=6   (= 2n)
down(n): if(n==0) return;
  print n; down(n-1);        down(3): 3 2 1
up(n): if(n==0) return;
  up(n-1); print n;          up(3): 1 2 3
test(n): if(n==0) return; print "A"+n;
  test(n-1); print "B"+n;
 test(2): A2 A1 B1 B2   test(3): A3 A2 A1 B1 B2 B3
sumDigits(n): if (n==0) return 0;
  return n%10 + sumDigits(n/10);
 (72) = 2 + (7 + 0) = 9
 iter: while (n>0) { s += n%10; n /= 10; }
countDigits(n): if (n<10) return 1;
  return 1 + countDigits(n/10);
 (472) = 1+(1+1) = 3     each call adds 1, not n
power(b,e): if (e==0) return 1;
  return b * power(b, e-1);
rev(s): if (s.length()<=1) return s;
  return rev(s.substring(1)) + s.charAt(0);
 rev("abc") = rev("bc")+a = (rev("c")+b)+a = "cba"
isPal(t): if (t.length()<=1) return true;
  if (t.charAt(0) != t.charAt(t.length()-1))
    return false;
  return isPal(t.substring(1, t.length()-1));
 "abba": a=a, "bb": b=b, "" true
 "abca": a=a, "bc": b!=c  false (2 calls)
 iter: i=0, j=t.length()-1;
  while (i<j) { if (t.charAt(i)!=t.charAt(j))
    return false; i++; j--; }  return true;
mystery(n): print n; mystery(n-1); print n
 mystery(3): 3 2 1 then 1 2 3 (base prints none)
f(n): if (n<=1) return 1; return f(n-1)+f(n-2);
 f(2)=2 f(3)=3 f(4)=5 ; calls for f(4) = 9
 isPal("level"): l=l, "eve": e=e, "v" true (3 calls)
TOH(n,start,goal,temp): if (n==0) return;
  TOH(n-1,start,temp,goal);
  print start --> goal;
  TOH(n-1,temp,goal,start);   call TOH(n,"A","C","B")
 n=2: A-B A-C B-C
 n=3 (7): A-C A-B C-B A-C B-A B-C A-C
 moves 2^n - 1
```

RAPID-FIRE ANSWERS (Java basics)

```
Arrays.toString(new int[3])        -> [0, 0, 0]
"Java".length()                    -> 4
"Hello".charAt(1)                  -> 'e'
int[] a={4,8,15}; a[3]             -> ArrayIndexOutOfBounds
int[] a = new int[3];              OK
int a[3];  int[] a = new int[];    do NOT compile
String s="hi"; char c = s[0];      does NOT compile
static int f(){ return null; }     does NOT compile
int x = null;                      does NOT compile
7 / 2 + " " + 7 % 2                -> 3 1
public Student(String name, int id){
  this.name = name; this.id = id; }
```

WRITE A RECURSIVE METHOD IN 4 STEPS

```
1 base case: smallest input, return directly
2 one piece of work: n%10, first letter, n
3 call yourself on a SMALLER input:
  n/10, n-1, substring(1)
4 combine: return piece + the call
sumDigits: base n==0 gives 0 | piece n%10
  | smaller n/10
palindrome: base length<=1 true | piece compare
  the ends | smaller substring(1, length-1)
A loop's stop condition = the recursive base case
```

LOOPS TO BIG-O

```
return a[5];                                  O(1)
for (i=0;i<n;i++) sum+=a[i];                  O(n)
for (...n) x++;  for (...n) y++;              O(n) two loops in a row
for (i=0;i<n;i++) for (j=0;j<n;j++)           O(n^2)
for (i=0;i<n;i++) for (j=0;j<i;j++)           O(n^2) (n(n-1)/2)
while (n > 1) { n = n/2; }                    O(log n)
T(n)=6n^2+30n+11                              O(n^2)
for (i=0;i<n;i++) for (j=0;j<5;j++)           O(n) inner is fixed
for (i=1;i<n;i*=2)                            O(log n) doubles
3 nested loops each to n                      O(n^3)
for (...n) x++; for (..n)for(..n) y++;        n+n^2 = O(n^2)
```

CSCI2010U MIDTERM - SIDE B: OOP, INTERFACES, BIG-O, LISTS, EXPLAIN WHY

INHERITANCE (L3, L4, Lab 2)

`extends`. Test "is a": Manager IS AN Employee, never the reverse. A subclass can add and **override**.

`super(...)` must be the FIRST line of the constructor (runs the parent constructor).

Parent `private` fields are not visible in the child: use the getter.

In an override, `super.m()` = parent version. Plain `m()` = itself, forever = **StackOverflowError** (runtime).

```
class Pet { private String name;
  public Pet(String name){ this.name = name; }
  public String getName(){ return name; }
  public String sound(){ return "..."; } }
class Dog extends Pet { private int tricks;
  public Dog(String name, int tricks){
    super(name);              // first line
    this.tricks = tricks; }
  @Override public String sound(){ return "Woof"; }
  public String describe(){
    return getName() + " knows " + tricks; } }
class Cat extends Pet { private int lives;
  public Cat(String name, int lives){
    super(name); this.lives = lives; }
  @Override public String sound(){ return "Meow"; }
  public String describe(){
    return getName() + " has " + lives; } }
class LoudDog extends Dog {
  public LoudDog(String n, int t){ super(n, t); }
  @Override public String sound(){
    return super.sound() + "!"; } }
Pet[] p = { new Pet("A"), new Dog("B",1),
            new LoudDog("C",2) };
for (Pet x : p) System.out.println(x.sound());
// prints:  ...   Woof   Woof!
new Dog("Rex",3).describe()  gives  Rex knows 3
```

```
class Manager extends Employee {
  private double bonus;
  public Manager(double salary, String name){
    super(salary, name); this.bonus = 0; }
  @Override public double getSalary(){
    return super.getSalary() + bonus; } }
// salary 70000, bonus 500 gives 70500
LimitedCounter(String name, int limit){
  super(name); this.limit = limit; }
@Override public void increment(){
  if (getCount() < limit) super.increment(); }
for (Counter c : counters) c.increment();
```

POLYMORPHISM (L4)

A parent variable can hold a child object. The version that RUNS is the object's own class (dynamic dispatch).

Read right to left with "is a". A parent variable cannot call child-only methods.

Making an object: declare a REFERENCE (an address), create the object with `new` on the **HEAP**, link them.

code	result
`Pet p = new Dog("B",1);`	OK (Dog is a Pet)
`p.sound()`	runs Dog's: Woof
`p.describe()`	COMPILE error (Pet has none)
`Dog d = new Pet("A");`	COMPILE error
`Pet[] arr`	holds Pet, Dog, Cat
`void feed(Pet x)`	accepts any Pet child

Polymorphism with an array (L4)

```
Manager boss = new Manager(95000, "Bill");
boss.setBonus(2750);
Employee[] w = { boss, new Employee(20745,"Peter"),
   new Employee(25000,"Mario") };
for (Employee e : w)
  println(e.getName() + ": $" + e.getSalary());
// Bill: $97750.0 (95000+2750) Peter: $20745.0 Mario:
$25000.0
```

COMPILE OR RUNTIME?

situation	error
override calls itself (no `super.`)	RUNTIME StackOverflowError
`a[4]` on length 4	RUNTIME ArrayIndexOutOfBounds
list `get` with a bad index	RUNTIME IndexOutOfBounds
`new Shape()` (abstract)	COMPILE
concrete class missing an abstract method	COMPILE
child reads parent's private field	COMPILE
`Dog d = new Pet("A");`	COMPILE
child-only method on a parent variable	COMPILE
`return null` in an int method	COMPILE

ABSTRACT CLASS (L5, Lab 3)

```
abstract class Shape {
  public abstract double area(); }   // no body
class Square extends Shape { private double s;
  public Square(double s){ this.s = s; }
  @Override public double area(){ return s*s; } }
new Shape()                          ERROR (abstract)
Shape b = new Square(2);             OK
class Triangle extends Shape { }     ERROR: no area()
 fix: write area() OR "abstract class Triangle"
```

```
abstract class Person { private String name;
  public Person(String name){ this.name = name; }
  public String getName(){ return name; }
  public abstract String getPosition(); }
class Employee extends Person { private double salary;
  public Employee(double salary, String name){
    super(name); this.salary = salary; }
  @Override public String getPosition(){
    return "Employee " + getName() + " on salary $" +
salary; } }
class Intern extends Person {
  public Intern(String name){ super(name); }
  @Override public String getPosition(){
    return "Intern unpaid"; } }
Person[] staff = { new Employee(20000,"Peter"),
                   new Intern("Tom") };
for (Person p : staff) println(p.getPosition());
 // Employee Peter on salary $20000.0  /  Intern unpaid
```

Cannot instantiate an abstract class, even with a constructor. A constructor can NOT be abstract. Mixes concrete + abstract methods; can have static methods.

A child must write ALL abstract methods or be abstract itself. Non-abstract = **concrete**.

Use it when siblings share structure: Person (abstract getPosition) with Employee and Intern, each writing its own.

INTERFACE AND compareTo

```
class Student implements Comparable<Student> {
  private String name;
  public Student(String name){ this.name = name; }
  @Override public int compareTo(Student o){
    return name.compareTo(o.name); } }   // Strings
class Book implements Comparable<Book> {
  private int pages;
  @Override public int compareTo(Book o){
    return Integer.compare(pages, o.pages); } } // numbers
class Employee extends Person
    implements Comparable<Employee> { ... }
Arrays.sort(arr); // Maya,Alex,Priya gives Alex,Maya,Priya
```

Returns **negative** if this comes first, **0** if equal, **positive** if after. `Arrays.sort` uses it; order does NOT come from insertion order.

Interface = a list of promises (a ROLE); all methods abstract. `implements` MANY, `extends` only ONE (diamond problem: two parents, which method?).

Generics `<T>` restrict the type: `ArrayList<String>` holds Strings only.

ACCESS AND PACKAGES

access	public	protected	none	private
same class	Y	Y	Y	Y
same pkg, subclass	Y	Y	Y	N
same pkg, other	Y	Y	Y	N
diff pkg, subclass	Y	Y	N	N
diff pkg, other	Y	N	N	N

Package: first line `package x;`, class names unique per package, no statement = default package, `java.lang` (System, Math, String) auto-imported, `java.util` has Arrays and Random. Course uses mostly public and private.

EXPLAIN WHY

Equal to target counts: "at least" means >=.

super(name) vs super.increment(): the first sets the parent's private state (first line only); the second calls the parent version of an overridden method.

new Person() fails: abstract, cannot be instantiated, even with a constructor.

Concrete class writes every abstract method: otherwise an object would have a method with no body.

Comparable and Arrays.sort: sort calls compareTo to decide the order. implements = many roles; extends = one parent.

Loop stop vs base case: both are the condition where work ends, and every step moves toward it.

Why private fields? Only methods can change state, so the class controls how its data is read and changed (encapsulation).

Why no two parents? Diamond: which inherited field or method wins? Interfaces avoid it because every method is abstract.

Palindrome mismatch: one differing outer pair already proves it is not one, so return false at once.

ALGORITHMS AND BIG-O (L6 to L9)

Algorithm = finite, deterministic (same input, same output), effective, implementable in any language. Pseudocode is for humans. **Time** = count steps; **space** = memory; time matters most.

T(n) = steps for input size n. Each line costs a constant; `end` and comments not counted. A loop header runs ONE MORE time than its body.

Linear search: best (first spot) O(1), worst (missing) `an+b` = O(n). We use **worst case**: an upper-bound guarantee that happens often. Average case not covered.

Big-O is a notation. Worst case is a type of input. Big-O can describe either.

Ladder, slow to fast: **1, log n, sqrt n, n, n log n, n^2, n^3, 2^n, n!**

Definition: f(n) = O(g(n)) if there are positive c and n0 with 0 <= f(n) <= c*g(n) for all n >= n0. g is an UPPER bound; O(g) is a SET.

Drop constants and small terms: 4n^2+20n+9 = O(n^2); 3+5n^2+7n^3 <= 15n^3 so O(n^3).

True/False rule: simplify, then LEFT = O(RIGHT) is true if left is the same or EARLIER on the ladder.

claim	answer
5n+20 = O(n)	T
n^2 = O(n)	F
n = O(n^3)	T
2^n = O(n^3)	F
3n+4 = O(n^2)	T
n log n = O(n^2)	T
3^n = O(n^3)	F
n^3 = O(3^n)	T
3+5n+18n^2 = O(n) / O(n^2)	F / T

T(n) AND BIG-O, WORKED

```
linear search lines: 1 for: c1(t+1) | 2 if: c2*t
  3 return i: c3 | 6 return -1: c4   (t = body runs)
best t=1: 2*c1 + c2 + c3 = constant = O(1)
worst t=n: c1(n+1) + c2*n + c4
  = (c1+c2)n + c1 + c4 = O(n)
perfect square v0 worst: about 4n steps (1 multiply,
  2 compares, 1 add per pass) = O(n)
3n+5 = O(n): 3n+5 <= 8n for n>=1  (c=8, n0=1)
n^2+n = O(n^2): n^2+n <= 2n^2 for n>=1  (c=2, n0=1)
3+5n^2+7n^3 <= 3n^3+5n^3+7n^3 = 15n^3  (c=15)
```

LISTS (L11)

Ordered = each element has a position (not sorted by value). Head = start, tail = end.

Interface says WHAT (add, addLast, get, isEmpty, size, toString); the implementation says HOW (ArrList uses an array).

`size` = stored now, `maxSize` = capacity. A bad index throws **IndexOutOfBoundsException** (unchecked, runtime). The slide checks `index > size`; the correct test is `index >= size`.

add at head = like array INSERT (shift right), O(n). addLast on a full list = shift left, O(n).

```
ArrList(){ this(DEFAULT_SIZE); }      // DEFAULT 10
ArrList(int capacity){ maxSize = capacity;
  size = 0; listArray = (T[]) new Object[capacity]; }
get(i): if (i<0 || i>=size)
  throw new IndexOutOfBoundsException("..");
  return listArray[i];
add(e,i) shift RIGHT (from the end):
  for (int k = size<maxSize ? size : maxSize-1;
       k > i; k--) listArray[k] = listArray[k-1];
  listArray[i] = e;  if (size<maxSize) size++;
addLast(e) when FULL, shift LEFT:
  for (int k = 1; k < listArray.length; k++)
    listArray[k-1] = listArray[k];
```

QUICK FACTS (past quiz)

Default int = **0**. `"Hello".length()` = **5**. `int[] arr = new int[5];` Last = `arr[arr.length-1]`.

Out of bounds = **ArrayIndexOutOfBoundsException**. Arrays fixed size. Java ArrayList remove last = O(1).

Constructor: `public Person(String name, int age) { this.name = name; this.age = age; }`

Printing an array directly gives garbage like [I@251a69d7; use `Arrays.toString(a)` (needs `import java.util.Arrays;`). `public static void main(String[] args)`.

Binary search needs a SORTED array. A method that sorts an array needs no return: arrays pass a reference.

Compile with `javac F.java`, run with `java F`. A Java `for` runs init once, then test, body, step.

Terms: class = blueprint, object = instance; object has state (fields), behaviour (methods), identity. Encapsulation = private fields + public methods. Nouns = classes, verbs = methods.